

宁波大学
NINGBO UNIVERSITY

实 验 报 告

[实验九 嵌入式文件 I/O 编译实验]

姓名：	赵瑞杰
学号：	216001944
课程名称：	嵌入式系统
所在学院：	信息科学与工程学院
专业班级：	21 级计算机科学与技术 1 班
任课教师：	王翀、王艺睿

实验项目名称	**实验九 嵌入式文件 I/O 编译实验**

一、实验目的与要求

1.1 实验目的

(1) 理解 Linux 系统中文件 I/O 操作的基本概念和原理。
(2) 学习使用标准 I/O 库函数进行文件的读写操作。
(3) 掌握文件描述符的使用，以及如何在用户空间通过系统调用来访问和操作文件。

1.2 实验要求

(1) 熟悉 Linux 系统中文件的分类，包括普通文件、目录文件、链接文件和设备文件。
(2) 编写程序实现对文件的打开、读写等基本操作，体会文件描述符在文件 I/O 中的作用。
(3) 通过实验加深对虚拟文件系统的理解，了解其在 Linux 系统中的工作原理和作用。
(4) 掌握基于文件描述符的 I/O 操作，包括但不限于 read、write、lseek 等系统调用的使用。

二、实验环境

(1) 华清远见开发环境。
(2) 实验平台为华清远见提供的虚拟机环境。
(3) 使用的操作系统是 Ubuntu 12.04 LTS 64-bit。
(4) 虚拟机工具软件为 VMware Player。

三、实验过程、步骤及内容

3.1 标准 I/O 实验

将“华清远见-CORTEXA9 资料： \实验代码\2、Linux 移植驱动及应用\1、Linux 应用\实验代码\06. Linux 系统标准 IO 实验\实验代码”目录下的“61.c”拷贝到【~/workdir/linux/application/8-io】目录下并编译：

```
$ gcc 61.c -o io_test
$ ./io_test
```

实验代码如下所示：

```
#include <stdio.h>
int main()
{
    int i = 0;
    for (i = 0; i < 379; i++)//每次向缓冲区内写三个字符
    {
        if (i >= 100)
            fprintf(stdout, "%d", i);
        else if (i >= 10)
            fprintf(stdout, "0%d", i);
        else if (i >= 0)
            fprintf(stdout, "00%d", i);
    }
    while (1);//强制执行，如果取消，程序结束时将会输出所有字符，看不到效果了。
}
```

3.2 Linux 系统文件目录操作编程实验

运行 ubuntu 系统，打开命令行终端。
执行命令,创建实验目录：

```
$ cd ~/workdir/linux/application/8-io
$ mkdir -p dir/example
$ cd dir/example
$ touch 1.c 2.c //在 example 目录下创建两个文件，后面程序会读这两个文件
$ cd ../
```

将“华清远见-CORTEXA9 资料： \实验代码\2、Linux 移植驱动及应用\1、Linux 应用\实验代码\07. Linux 系统文件目录操作编程实验\实验代码\7.2”目录下的“myls.c”文件拷贝到该目录下。

执行命令：

```
$ gcc myls.c -o myls
$./myls example
```

实验代码如下：

```c
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <errno.h>
#include <unistd.h>
#include <string.h>
#include <time.h>
#include <pwd.h>
#include <grp.h>
#include <dirent.h>

#define N 128

int aflag = 0, lflag = 0;

void displayfile(char *name, char *showname);
void displaydir(const char *name);

int main(int argc, char *argv[])
{
    struct stat buf;
    int ch, i;

    if (argc < 2)
    {
        fprintf(stdout, "usage: %s filename\n", argv[0]);
        exit(-1);
    }

    while ((ch = getopt(argc, argv, "al")) != -1)
```

```
        {
            switch(ch)
            {
            case 'a':
                aflag = 1;
                break;
            case 'l':
                lflag = 1;
                break;
            default:
                printf("wrong option %c\n", optopt);
            }
        }

        //  printf("aflag=%d lflag=%d\n", aflag, lflag);

        for (i = optind; i < argc; i++)
        {
            if (stat(argv[i], &buf) == -1)
            {
                perror("stat");
                exit(-1);
            }

            if (!S_ISDIR(buf.st_mode))
                displayfile(argv[i], argv[i]);
            else
                displaydir(argv[i]);
        }
        if (lflag == 0)
            printf("\n");
        return 0;
    }

    void displayfile(char *name, char *showname)
    {
        struct stat buf;
        struct tm *p;
        int i = 8;

        if (lflag == 0)
        {
            printf("%s ", showname);
            return;
```

```
        }

        if (stat(name, &buf) == -1)
        {
            perror("stat");
            exit(-1);
        }

        switch (buf.st_mode & S_IFMT)
        {
        case S_IFSOCK:
            printf("s");
            break;
        case S_IFLNK:
            printf("l");
            break;
        case S_IFREG:
            printf("-");
            break;
        case S_IFBLK:
            printf("b");
            break;
        case S_IFDIR:
            printf("d");
            break;
        case S_IFCHR:
            printf("c");
            break;
        case S_IFIFO:
            printf("p");
            break;
        default:
            printf("?");
        }
    while (i >= 0)
        {
            if ((buf.st_mode) >> i & 1)
            {
                switch (i % 3)
                {
                case 2:
                    printf("r");
                    break;
                case 1:
```

```
                    printf("w");
                    break;
                case 0:
                    printf("x");
                    break;
                }
            }
            else
            {
                printf("-");
            }
            i--;
        }

        printf(" ");

        /* hard links */
        printf("%d", buf.st_nlink);

        /* uid gid */
        printf(" %s %s  ", getpwuid(buf.st_uid)->pw_name, getgrgid(buf.st_gid)->gr_
    name);

        printf("%ld  ", buf.st_size);

        p = localtime(&buf.st_mtime);
        printf("%d-%d-%d %d:%d  ", p->tm_year + 1900, p->tm_mon + 1,
               p->tm_mday, p->tm_hour, p->tm_min);
        printf("%s\n", showname);
    }
    void displaydir(const char *name)
    {
        DIR *dir;
        struct dirent *diritem;
        char str[N];

        dir = opendir(name);
        while ((diritem = readdir(dir)) != NULL)
        {
            if (strncmp(".", diritem->d_name, 1) == 0 && aflag == 0)
                continue;
            sprintf(str, "./%s", diritem->d_name);
            displayfile(str, diritem->d_name);
        }
```

```
}
```

四、实验结果

4.1 标准 I/O 实验

（1）创建 8-io 文件夹，拷贝文件到 8-io 中，进入文件夹：

```
linux@ubuntu64-vm:~$ cd workdir/linux/application/
linux@ubuntu64-vm:~/workdir/linux/application$ sudo mkdir 8-io
[sudo] password for linux:
linux@ubuntu64-vm:~/workdir/linux/application$ cp /mnt/hgfs/share/实验代码/06.\
Linux系统标准IO实验/实验代码/6
61.c  64/
linux@ubuntu64-vm:~/workdir/linux/application$ sudo cp /mnt/hgfs/share/实验代码/
06.\ Linux系统标准IO实验/实验代码/61.c 8-io/ -a
linux@ubuntu64-vm:~/workdir/linux/application$ cd 8-io
```

（2）运行对应程序，结果如下所示：

```
linux@ubuntu64-vm:~/workdir/linux/application/8-io$ sudo gcc 61.c -o io_test
linux@ubuntu64-vm:~/workdir/linux/application/8-io$ ./io_test
000001002003004005006007008009010011012013014015016017018019020021022023024025026
602702802903003103203303403503603703803904004104204304404504604704804905005105206
530540550560570580590600610620630640650660670680690700710720730740750760770780790
080081082083084085086087088089090091092093094095096097098099100101102103104105106
610710810911011111211311411511611711811912012112212312412512612712812913013113213
331341351361371381391401411421431441451461471481491501511521531541551561571581591
160161162163164165166167168169170171172173174175176177178179180181182183184185186
618718818919019119219319419519619719819920020120220320420520620720820921021121221
132142152162172182192202212222232242252262272282293023123223323423523623723823923
240241242243244245246247248249250251252253254255256257258259260261262263264265266
626726826927027127227327427527627727827928028128228328428528628728828929029129229
932942952962972982993003013023033043053063073083093103113123133143153163173183193
320321322323324325326327328329330331332333334335336337338339340341342343344345346
```

可以看到终端屏幕共输出了 1024 字节大小的内容，多余的内容没有输出，这就验证了全缓存的特点：当填满 I/O 缓存后才进行实际的 I/O 操作。

4.2 Linux 系统文件目录操作编程实验

（1）创建文件夹，并创建 1.c，2.c 两个文件:

```
linux@ubuntu64-vm:~/workdir/linux/application/8-io$ sudo mkdir -p dir/example
linux@ubuntu64-vm:~/workdir/linux/application/8-io$ cd dir/example
linux@ubuntu64-vm:~/workdir/linux/application/8-io/dir/example$ sudo touch 1.c 2.c
linux@ubuntu64-vm:~/workdir/linux/application/8-io/dir/example$ cd ..
```

（2）拷贝代码到文件夹中：

```
linux@ubuntu64-vm:~/workdir/linux/application/8-io$ sudo cp /mnt/hgfs/share/实验代码/07.\ Li
nux系统文件目录操作编程实验/实验代码/7.2/myls.c dir/ -a
linux@ubuntu64-vm:~/workdir/linux/application/8-io$ cd dir
linux@ubuntu64-vm:~/workdir/linux/application/8-io/dir$ ls
example  myls.c
```

（3）执行对应的文件，操作结果如下：

```
linux@ubuntu64-vm:~/workdir/linux/application/8-io/dir$ sudo gcc myls.c -o myls
myls.c: 在函数‘displayfile’中:
myls.c:114: 警告： 格式‘%d’需要类型‘int’，但实参 2 的类型为‘__nlink_t’
linux@ubuntu64-vm:~/workdir/linux/application/8-io/dir$ ./myls example
2.c 1.c
linux@ubuntu64-vm:~/workdir/linux/application/8-io/dir$
```

五、实验结论

在“嵌入式文件 I/O 编译实验”中，我学到了 Linux 系统中关于文件 I/O 操作的基本概念和原理，深入理解了标准 I/O 库函数在文件读写操作中的应用。通过实验，我掌握了文件描述符的使用，了解了如何在用户空间通过系统调用来访问和操作文件。此外，我还熟悉了 Linux 系统中文件的分类，包括普通文件、目录文件、链接文件和设备文件，体会到了文件描述符在文件 I/O 中的重要作用。这些知识增强了我对 Linux 系统文件操作的理解，为后续的嵌入式系统开发奠定了坚实的基础。

在实验过程中，我成功编写并运行了程序，实现了对文件的打开、读写等基本操作。通过实验，我对虚拟文件系统的工作原理有了更深入的了解，特别是其在 Linux 系统中的作用。在实际操作中，我使用 read、write、lseek 等系统调用进行基于文件描述符的 I/O 操作，进一步加深了对这些系统调用的理解和掌握。这次实验不仅提高了我的编程技能和解决问题的能力，还增强了我在 Linux 环境中进行文件操作的信心。